2023 3rd Asian Conference on Innovation in Technology (ASIANCON)
Pune, India. Aug 25-27, 2023

Malicious URLs and QR Code Classification using Machine Learning and Deep Learning Techniques

Delano Oscar Do Rosario Lourenco
School of Computer Science & Engineering
Vellore Institute of Technology
Chennai,Tamil Nadu, India
ORCID : 0009-0009-1345-1031

M V H Sai Sriraj
School of Computer Science & Engineering
Vellore Institute of Technology
Chennai,Tamil Nadu, India
ORCID : 0009-0009-6716-3772

Krotha Kayne Thambi
School of Computer Science & Engineering
Vellore Institute of Technology
Chennai,Tamil Nadu, India
ORCID : 0009-0001-6905-8767

Vibho Ranjan
School of Computer Science & Engineering
Vellore Institute of Technology
Chennai,Tamil Nadu, India
ORCID : 0009-0000-6403-2675

2023 3rd Asian Conference on Innovation in Technology (ASIANCON) | 979-8-3503-0228-8/23/$31.00 ©2023 IEEE | DOI: 10.1109/ASIANCON58793.2023.10270125

***Abstract*— Attackers commonly employ malicious URLs and harmful QR codes to spread malware and phishing scams; therefore, they need to be classified. Malicious URLs that lead visitors to phishing or malware-infected websites that steal personal data can be included in emails, social media posts, or website content. Rogue QR codes can be used to propagate malware, steal data, or direct users there, much like malicious websites can.**

In this paper, the classification of malicious content is provided using two approaches:(1) Malware URL categorization based on ML and malware QR codes (2) classification of malicious QR codes based on deep learning.

The first technique classifies URLs as dangerous or benign using machine learning models that are trained on features derived from the URLs. Several ML techniques, such as Random Forest, Naive Bayes, and Support Vector Machine, are used to evaluate the model's performance.

The second option focuses on classifying harmful QR codes using deep learning techniques. The classification of QR codes as malicious or benign is done using CNN and well-known transfer learning models like RESNET.

In general, the offered approaches provide effective techniques to classify unsafe material, which can be utilized to enhance security.

Keywords—Machine Learning, Malicious QR code, Malicious URLs, Deep Learning, Transfer Learning, Classification

I. Introduction

Malicious URLs and malicious QR codes must be categorized because attackers frequently use them to disseminate malware and phishing scams. Malicious URLs can be inserted into emails, social media posts, or website content and direct users to phishing or malware-infected websites that steal personal data. Similar to malicious websites, malicious QR codes can be used to spread malware, steal data, or lead users there.

It is important to categorize malicious URLs and QR codes because doing so makes it possible to identify and stop these threats. Security systems can stop users from accessing malware-infected websites or falling for phishing scams by identifying and blocking malicious URLs and QR codes. The overall security of systems and networks can be improved by using classification to train ML models to recognize new and emerging threats.

A. Algorithms Used

1) Random Forest

To increase the accuracy of the data, a classification system called random forest uses multiple decision trees on different parameters of the input data. It is based on the concept of ensemble learning, which is the process of combining multiple classifiers to solve complex problems and improve model performance. Hyperparameters for random forests are very similar to decision trees or bagging classifiers. Using the algorithm's regressor, you can use random forests for regression functions. Random forests add more randomness to the model as the trees grow. When dividing a node, it looks for the best features in a very random subset of features, not the most important features. Because of the diversity this problem creates, it often leads to better models.

2) Decision Tree

A decision tree represents the outcome of various decisions. Generally, a decision tree has a starting point that leads to a conclusion. Each of these events is linked to additional nodes, which are divided into additional probabilities. Therefore it is like a tree. Circle-based luck nodes show the probability of various events. Final nodes show the outcome of the decision, and decisions (shown by squares) show the decisions that need to be made.

3) Logistic Regression

Logistic regression is used to predict a categorical dependent variable using a set of independent variables. Logistic regression estimates the output of the categorical dependent variable. Therefore, the result must be categorical or discrete. It can be a value between 0 and 1. In logistic regression, instead of fitting a straight line, we fit an "S" shaped logistic function, which is usually a function that maps the predicted values to probability (i.e., 0 to 1).

4) Naïve Bayes

Authorized licensed use limited to: UNIVERSITI TEKNOLOGI MARA. Downloaded on April 28,2025 at 03:36:16 UTC from IEEE Xplore. Restrictions apply.

One of the popular probabilistic ML approach for classification problems is Naive Bayes. It is predicated on the Bayes theorem, a mathematical method for estimating the likelihood of a hypothesis in light of existing information and fresh data. The method initially creates a probability model based on the training data by calculating the likelihood of each class given the characteristics of the data. The program then uses Bayes' theorem to determine the likelihood of each class given the characteristics of fresh data that has to be categorized. The data is then assigned to the class with the highest likelihood.

5) XGBoost

XGBoost (Extreme Gradient Boosting) is a ML algorithm that is used for regression, classification, and ranking problems. It is based on the gradient boosting framework. XGBoost is designed to be efficient, scalable, and accurate, making it a popular choice for many ML applications. The algorithm works by building a series of decision trees, where each new tree tries to correct the mistakes of the previous trees. It does this by placing more emphasis on the data points that were incorrectly classified by the previous trees. XGBoost uses a technique called gradient descent to optimize the parameters of the decision trees and reduce the loss function, which measures the difference between the predicted values and the actual values.

6) CNN

CNN (Convolutional Neural Network) is a form of neural network that is frequently used for image and video recognition tasks in the context of deep learning.

Convolutional, pooling, and fully connected layers are only a few of the layers of interconnected nodes that make up CNNs. The pooling layers reduce the dimensionality of the output from the convolutional layers, while the convolutional layers accomplish feature extraction by applying a collection of filters to the input image. The categorization is subsequently carried out by the fully linked layers at the network's end using the extracted features.

Numerous image and video identification tasks, including object detection, face recognition, and image segmentation, have demonstrated the excellent effectiveness of CNNs. They have also been utilized in other applications, including speech recognition and natural language processing.

CNNs have revolutionized several disciplines, including computer vision, and allowed machines to perform some jobs with accuracy comparable to that of humans. They are an essential component of the deep learning toolset, and the scope of their use is growing quickly.

7) Resnet Model

Kaiming He et al. developed the ResNet (Residual Network) deep learning architecture in 2015 to tackle the issue of vanishing gradients by making use of residual connections.

In a ResNet, an input layer's output is combined with it rather than fully replacing it. This establishes a "residual" connection that, even in extremely deep topologies, makes it easier for gradients to move across the network. ResNets have demonstrated outstanding performance in a range of computer vision applications, including semantic segmentation, object detection, and image classification.

Advantages of Resnet Model include: In very deep neural networks, where conventional CNNs may experience vanishing gradients, ResNets perform exceptionally well. On a variety of computer vision applications, they achieve state-of-the-art performance, frequently outperforming competing CNN systems. ResNets may be trained on big datasets with only a few parameters and are generally simple to train. They are extremely scalable and adaptable for a variety of applications.

Disadvantages of Resnet Model include: The cost of computing to train ResNets can be high, particularly for very deep architectures. As with any deep learning architecture, they could be prone to overfitting on small datasets. In comparison to conventional CNNs, the network may be more challenging to decipher and comprehend due to the utilization of residual connections. Overall, ResNet is a deep learning architecture that is quite successful and is frequently used in computer vision applications. Although it has significant drawbacks, its benefits make it a strong tool for a variety of purposes.

8) VGG19 Model

The Visual Geometry Group (VGG) at the University of Oxford created the deep neural network architecture VGG19 in 2014. With extra convolutional layers, it is a variation of the VGG16 architecture, which was also created by the VGG group.

A total of 19 layers make up the VGG19 architecture, including 16 convolutional layers and 3 fully linked layers. It is a deep convolutional neural network that is effective at several computer vision tasks, including semantic segmentation, object identification, and image classification.

All of the convolutional layers of the VGG19 architecture use tiny 3x3 filters, which enables a more thorough and localized feature extraction procedure. To decrease the dimensionality of the feature maps and prevent overfitting, the network uses max pooling layers.

Benefits of VGG19: On a range of picture recognition tasks, including the ImageNet Large Scale Visual Recognition Challenge, VGG19 achieves state-of-the-art performance.
VGG19 is very good at capturing fine-grained features in images because of the use of modest 3x3 filters in all of its convolutional layers.

VGG19's design is fairly straightforward to comprehend, making it a suitable place to start for people who are new to deep learning.

Negative aspects of VGG19: VGG19 has a lot of layers, which makes training computationally expensive and deployment challenging for devices with limited resources. On some tasks, VGG19 might not perform as well as more modern neural network architectures like ResNet or Inception.

The accuracy of some tasks, including semantic segmentation, may be hampered by the employment of max pooling layers because of the potential loss of spatial information.

VGG19 is a benchmark for image recognition tasks since it is a very powerful deep neural network architecture. Although it has significant drawbacks, its benefits make it an effective tool for a variety of computer vision applications.

Authorized licensed use limited to: UNIVERSITI TEKNOLOGI MARA. Downloaded on April 28,2025 at 03:36:16 UTC from IEEE Xplore. Restrictions apply.

II. LITERATURE SURVEYS

[1] This paper deals with the issue of identifying and categorizing harmful URLs in emails and websites, which pose serious hazards to businesses. Using the ISCX-URL-2016 dataset, the study compares the effectiveness of deep learning frameworks and conventional ML methods in identifying and categorizing harmful URLs. According to the results, the accuracy of the Random Forest, Keras-TensorFlow, and Fast.ai models was over 96%, and Random Forest was shown to be the best model overall in terms of time, performance, and complexity restrictions. In addition, the study discovered that, as compared to employing all of the dataset's characteristics, using just the top 5–10 features produced the greatest results.

[2] This study focuses on how hostile examples might cause disturbed input samples to cause ML models to provide incorrect results. The paper offers two techniques that make use of QR codes to check the visual accuracy of image-based datasets and find any anomalies. To determine if a picture has been altered, the first technique maintains a verification string for each image in the dataset, whereas the second method just keeps a single verification string.

[3] This paper covers the growing danger of network information insecurity and how attackers use malicious URLs along with end-to-end technology and human vulnerabilities. Based on a new collection of URL attributes and behaviors, the study suggests a malicious URL detection method that makes use of ML methods and big data technology. The results of the experiments demonstrate that the suggested approach can considerably increase the capability of identifying harmful URLs, making it an efficient and user-friendly way to detect malicious URLs.

[4] This study explores the security concerns surrounding QR codes and suggests a machine learning-based technique for spotting fraudulent QR codes. In addition to being used to visit websites and make purchases, QR codes can also be used for nefarious purposes like phishing and pharming. The identification of malicious QR codes is examined in the study as a two-way classification problem, and various ML algorithms, including K-Nearest Neighbor, Random Forest, Support Vector Machine, and Binary LSTM are tested for their efficacy. The suggested approach is determined to be a safe and practical solution for QR code security.

[5] Information security is seriously threatened by malware, a class of software that is more common and sophisticated that is intended to damage computer systems or networks. Information security now requires the detection and classification of malware. Malware classification is the procedure of classifying malware into several categories according to its actions and traits. Malware composition analysis, on the other side, examines the features and actions of a malware sample. The purpose is to learn more about the motives of the attacker and the infection.

For malware categorization and composition analysis, many methods have been put out recently, ranging from signature-based approaches to methods based on ML. Although these methods have progressed, malware creators continue to create new evasion strategies to prevent detection. To increase malware detection and analysis, it is important to stay current with this field of study.

This study tries to categorize and contrast the key conclusions from malware composition and classification studies. Also, we go over malware evasion strategies and feature extraction approaches. Also, we describe each reviewed research based on the characteristics and algorithms utilized, highlighting their advantages and disadvantages. Lastly, we discuss problems, roadblocks, and potential future research directions in malware analysis.

Classification of Malware: Malware classification is the process of classifying malware samples according to their behavior and traits into various kinds. The most popular techniques for classifying malware include signature-based detection, heuristics-based detection, and ML-based detection.

A malware sample is compared to a database of known malware signatures in signature-based detection. The sample is labeled as malware if it fits one of the signatures. Although quick and effective, this method has several drawbacks, such as the inability to identify brand-new viruses.

Heuristics-based detection includes examining a program's behavior to determine whether it is malicious. Compared to signature-based detection, this technique is more sophisticated, but it is also more complicated and prone to false positives.

With machine learning-based detection, lots of data are analyzed using algorithms to find patterns and categorize malware samples. This technique can identify new and unidentified viruses, but it needs a lot of data to adequately train the algorithm.

Malware Analysis by Composition: Analysis of the functionalities and behaviors of a malware sample is known as malware composition analysis. The purpose is to learn more about the motives of the attacker and the infection. The most popular techniques for analyzing the composition of malware are dynamic analysis and static analysis.

Malware Avoidance Methods: Obfuscation, polymorphism, and metamorphism are only a few of the methods used by malware developers to avoid detection. Obfuscation includes burying a malware sample's source code to make it more difficult to find. Polymorphism is the process of altering a malware sample's code to produce new, more evasive forms. A malware sample can metamorphosize by altering its structure and behavior to produce new, harder-to-detect variations.

Techniques for Feature Extraction: The technique of choosing and extracting pertinent features from a malware sample for use in composition and classification analysis is known as feature extraction. API calls, system calls, and file system activities are examples of frequently utilized functionalities. The approaches for feature selection and extraction include deep learning-based feature extraction, automatic feature selection, and manual feature selection.

Conclusion: Given the intricacy of malware creation and current advancements in communication and computer infrastructure, malware detection and categorization are becoming more difficult. The current malware classification methods allow reverse engineers to comprehend

[6] Malware proliferation has raised serious concerns in recent years, making it essential to find and eliminate these dangerous agents. This study suggests using a deep learning-

Authorized licensed use limited to: UNIVERSITI TEKNOLOGI MARA. Downloaded on April 28,2025 at 03:36:16 UTC from IEEE Xplore. Restrictions apply.

based SACNN to classify 25 well-known malware types based on images. On the dataset introduced in [13], the proposed model was assessed using several evaluation criteria, including precision, recall, specificity, and F1 score. High scores on all of the evaluation indicators were found for the suggested model with class balancing, which demonstrated its excellent performance. It is also interesting that the suggested model can perform well even when the benign class is balanced.

The suggested model takes a novel method to classify image-based malware by utilizing spatial attention and a convolutional neural network. The findings of this study show that even in the face of cutting-edge, automated malware production techniques, the suggested approach is highly effective at spotting and classifying malware images. The paper contends that the proposed model can be less complex than existing options while yet providing equivalent or superior performance.

Overall, the results of this study have significant implications for the cybersecurity sector and show how deep learning techniques might help with problems brought on by the growing threat of malware. The suggested model can be applied to other malware detection tasks in future research, and its effectiveness can be assessed using larger and more varied datasets.

[7] Malware mitigation has been a long-standing goal of the information security community. The creation of an anti-malware system that can combat unidentified malware is a common endeavor that could be advantageous to many industries.

The authors hope to develop an intelligent deep learning (DL)-based anti-malware system. By applying such models, it would be possible to mathematically generalize the detection of recently released malware. Identifying the connection between a specific malware x and the malware family that it belongs to, f:xy.

The Malimg dataset (Nataraj et al., 2011) of malware pictures extracted from malware binaries was employed by the authors to achieve this accomplishment. The authors then trained the CNN-SVM (Tang, 2013), GRU-SVM (Agarap, 2017), and MLP-SVM DL models 1 to categorize each malware family. The GRU-SVM stands out among the DL models with a prediction accuracy of 84.92%, according to empirical data.

This makes sense given that, of the models shown, the aforementioned model had the comparably most complex architectural design. The next step in creating an intelligent anti-malware system is to investigate an even more ideal DL-SVM model.

[8] ML techniques have been applied to the challenging challenge of malware categorization for many years. However, development has frequently been sluggish, in part because of a variety of particular challenges with the task that arise during all phases of creating a ML system: data collection, labeling, feature generation and selection, model selection, and evaluation. The authors discussed a number of the current approaches and difficulties in malware classification in this survey, including data collection, feature extraction, model design, and model evaluation. The authors talk about limitations that must be taken into account for machine learning-based solutions in this field as well as unsolved issues that machine learning might also be able to help with. This survey intends to be helpful to cybersecurity practitioners who want to learn more about how machine learning may be used to address the malware problem, as well as to provide data scientists with the background they need to understand the difficulties in this particularly challenging field.

[9] Due to its accessibility and adaptability, Android has become the dominant mobile ecosystem due to its broad adoption. That has, however, also turned it becomes the main target of malicious programs. Hence, the necessity for a reliable malware detection system is critical. This work suggests a transfer learning and malware visual features-based explainable malware detection system. The suggested method increases malware detection precision by making use of both textual and visual data.

A pre-trained BERT model is used to extract trained textual features. The malware-to-image conversion mechanism is additionally presented to render network byte streams visually. The Features from the Accelerated Segment Test extractor and Binary Robust Independent Elementary Features descriptor are used to effectively extract and mark key features. The proposed malware detection system's effectiveness is ensured by the combination of these techniques.

Despite its benefits, the suggested strategy has several drawbacks. The malware detection system's overall performance may be impacted by the inaccuracy of the conversion of network byte streams into visual representations. Also, the performance of the suggested strategy can be impacted by the quantity and caliber of training data. The final drawback of the suggested method is that it can need a lot of processing power and time for training and detection.

[10] The detection of obfuscated malware has been a topic of significant research interest in recent years as malicious actors continue to develop new techniques to evade detection. One such study focused specifically on Android applications and demonstrated that some malicious Android apps are re-released under new names and with obfuscated certificate information. To combat this, the researchers employed a novel approach that leveraged the correlation between package names and certificate owner names to classify applications as malicious or benign.

In their method, the researchers used stacking techniques to combine CNNs to analyze information obtained from the package name and certificate owner. The resulting classification system was able to accurately detect obfuscated malware in Android apps with high precision and recall. However, it should be noted that the features used in this study were specific to the Android operating system and cannot be extended to other operating systems, such as Linux or Windows.

While the study focused on the detection of obfuscated Android malware, the approach could be adapted to other types of malicious code and could potentially be extended to other operating systems with the development of new features. As the threat landscape continues to evolve, it is clear that the development of effective methods for detecting and classifying obfuscated malware will be crucial in ensuring the security of computer systems and networks.

Authorized licensed use limited to: UNIVERSITI TEKNOLOGI MARA. Downloaded on April 28,2025 at 03:36:16 UTC from IEEE Xplore. Restrictions apply.

III. RESEARCH GAP

Malicious URL detection using machine learning is a well-researched area, but there are still some gaps that need to be addressed. Some of the research gaps in malicious URL detection using machine learning are:

Lack of large and diverse datasets: The performance of machine learning models depends heavily on the quality and size of the datasets used to train them. While there are some publicly available datasets for malicious URL detection, they are often limited in size and scope, which can limit the generalizability of the resulting models.

Limited feature engineering: The practice of identifying and extracting pertinent features from data to enhance the performance of machine learning models is known as feature engineering. However, limited features are being considered for malicious URL detection, leading to inadequate efficiency of machine learning models.

Adversarial attacks on machine learning models: Adversarial attacks are a major threat to the robustness and effectiveness of ML models for malicious URL detection. Attackers can easily generate malicious URLs that can bypass the detection mechanism. Therefore, developing robust ML models that can withstand such attacks is a significant challenge.

Interpretability: Interpretability is the ability to explain how a ML model makes its decisions. This is important for trust and accountability, but many ML models for malicious URL detection are black boxes, making it difficult to understand how they arrive at their decisions.

Online learning: Malicious URL detection requires the ability to adapt to new types of attacks quickly. This is particularly challenging in the case of machine learning models, which typically require retraining when new data is added. Therefore, developing efficient online learning techniques is critical for the scalability of the system.

Existing models for malicious URL and QR code detection using deep learning typically use neural network-based architectures such as CNNs, RNNs, or their variants such as LSTM networks or GRUs.

For example, in malicious URL detection, the CNN-based model proposed by Zhang et al. (2021) acquired an accuracy of 98.63% on a [13]. In comparison, the LSTM-based model proposed by Ghiasi et al. (2020) acquired an accuracy of 97.22% on the same dataset.

In QR code detection, the Faster R-CNN model proposed by Lee et al. (2021) acquired an accuracy of 99.3%, while the YOLOv3 model proposed by Huang et al. (2020) acquired an accuracy of 99.07%.

Deep learning-based models have shown improved performance in both malicious URL and QR code detection compared to previous models using traditional machine learning techniques. This is because deep learning models can automatically learn relevant features from the data, which reduces the need for manual feature engineering. Additionally, deep learning models can capture complex patterns and relationships in the data, which may not be possible with traditional machine learning models.

However, deep learning models require large amounts of data and computational resources, which may limit their scalability in real-world scenarios. Moreover, deep learning models can be challenging to interpret, which is a significant concern in applications such as malicious URLs and QR code detection, where it is essential to understand how the models make decisions to ensure their trustworthiness.

In summary, deep learning-based models have shown improved performance in both malicious URL and QR code detection compared to previous models using traditional machine learning techniques. However, the scalability and interpretability of deep learning models remain a significant concern for real-world applications.

IV. PROPOSED METHODOLOGY

A. *Proposed Workflow*

The below workflow diagram shows the workflow of our research starting from the URL data collection from [11]. The study mainly follows two sub-branches the first one deals with the URL dataset directly by applying different Machine Learning models with hyper-parameter tuning, and the second major part of our study is based on the Deep Learning approaches we use to classify the malicious QR code images by converting the malicious URLs from the dataset obtained from [11] into QR code using a recurring operation in python and saving the QR code in the form of black and white image of size 224*224 .

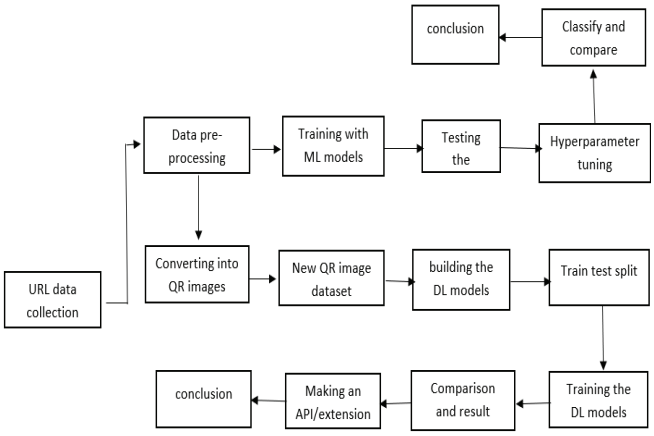

Fig. 1. Work Flow Diagram of our Project

B. *Dataset Used*

In this research study, we used the open source available dataset [11], which contains a malicious URL dataset, a parent class of benign or malware, and the subcategory of malware such as spam, malware, defacement, and phishing. The statistics and the source of different samples are as follows:

- Benign URLs: From the top Alexa websites, more than 35,300 benign URLs were gathered. To retrieve the URLs, the domains were run via a Heritrix web crawler. Initial crawling of about 500,000 distinct URLs is followed by a pass to weed out duplicate and domain-only URLs. The collected URLs were afterward run using Virustotal to remove any safe URLs.
- Spam URLs: The publicly accessible WEBSPAM-UK2007 dataset contained about 12,000 spam URLs.

Authorized licensed use limited to: UNIVERSITI TEKNOLOGI MARA. Downloaded on April 28,2025 at 03:36:16 UTC from IEEE Xplore. Restrictions apply.

- Phishing URLs: An active database of phishing sites called OpenPhish contained about 10,000 phishing URLs.
- Malware URLs: From DNS-BH, a project that maintains a list of malware websites, more than 11,500 URLs associated with malware websites were gathered.
- More than 45,450 URLs fall within the category of defacement URLs. They are reputable websites with high Alexa rankings that host harmful websites with disguised or false URLs.

Apart from this, we have created a QR code dataset based on the URLs from [11], which is further fitted to the deep learning models.

Fig. 2. shows the Benign class samples

Fig. 3. shows the defacement class samples

Fig. 4. shows the phishing class samples

Fig. 5. shows the spam class samples

Fig. 6. shows the malware class samples

C. Proposed Architecture Diagram of Deep Learning Models

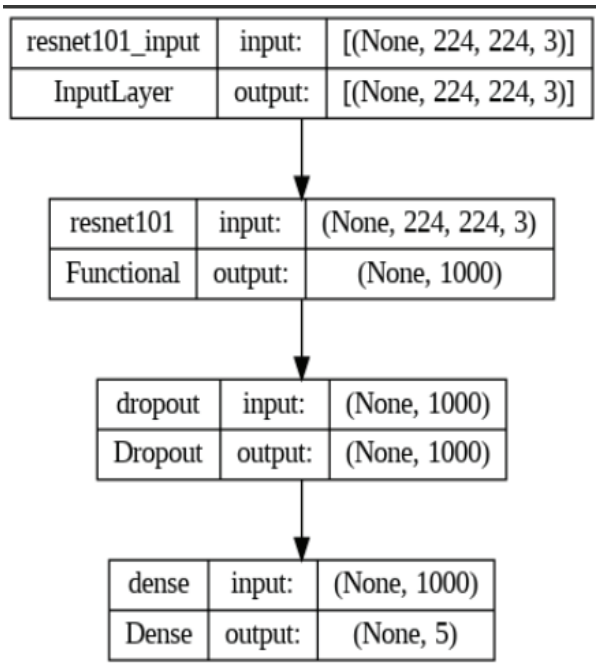

Fig. 7. Architecture Diagram of the Resnet 101 used

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d (Conv2D)             (None, 224, 224, 32)      896

 conv2d_1 (Conv2D)           (None, 224, 224, 32)      9248

 max_pooling2d (MaxPooling2D  (None, 112, 112, 32)     0
 )

 conv2d_2 (Conv2D)           (None, 112, 112, 64)      18496

 conv2d_3 (Conv2D)           (None, 112, 112, 64)      36928

 max_pooling2d_1 (MaxPooling  (None, 56, 56, 64)       0
 2D)

 conv2d_4 (Conv2D)           (None, 56, 56, 128)       73856

 conv2d_5 (Conv2D)           (None, 56, 56, 128)       147584

 max_pooling2d_2 (MaxPooling  (None, 28, 28, 128)      0
 2D)

 flatten (Flatten)           (None, 100352)            0

 dense (Dense)               (None, 128)               12845184

 dense_1 (Dense)             (None, 2)                 258

=================================================================
Total params: 13,132,450
Trainable params: 13,132,450
Non-trainable params: 0
_________________________________________________________________
```

Fig. 8. shows the proposed CNN Architecture

Authorized licensed use limited to: UNIVERSITI TEKNOLOGI MARA. Downloaded on April 28,2025 at 03:36:16 UTC from IEEE Xplore. Restrictions apply.

```
Model: "vgg19"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 input_1 (InputLayer)        [(None, None, None, 3)]   0

 block1_conv1 (Conv2D)       (None, None, None, 64)    1792

 block1_conv2 (Conv2D)       (None, None, None, 64)    36928

 block1_pool (MaxPooling2D)  (None, None, None, 64)    0

 block2_conv1 (Conv2D)       (None, None, None, 128)   73856

 block2_conv2 (Conv2D)       (None, None, None, 128)   147584

 block2_pool (MaxPooling2D)  (None, None, None, 128)   0

 block3_conv1 (Conv2D)       (None, None, None, 256)   295168

 block3_conv2 (Conv2D)       (None, None, None, 256)   590080

 block3_conv3 (Conv2D)       (None, None, None, 256)   590080

 block3_conv4 (Conv2D)       (None, None, None, 256)   590080

 block3_pool (MaxPooling2D)  (None, None, None, 256)   0

 block4_conv1 (Conv2D)       (None, None, None, 512)   1180160

 block4_conv2 (Conv2D)       (None, None, None, 512)   2359808

 block4_conv3 (Conv2D)       (None, None, None, 512)   2359808

 block4_conv4 (Conv2D)       (None, None, None, 512)   2359808

 block4_pool (MaxPooling2D)  (None, None, None, 512)   0

 block5_conv1 (Conv2D)       (None, None, None, 512)   2359808

 block5_conv2 (Conv2D)       (None, None, None, 512)   2359808

 block5_conv3 (Conv2D)       (None, None, None, 512)   2359808

 block5_conv4 (Conv2D)       (None, None, None, 512)   2359808

 block5_pool (MaxPooling2D)  (None, None, None, 512)   0

=================================================================
Total params: 20,024,384
Trainable params: 0
Non-trainable params: 20,024,384
_________________________________________________________________
```

Fig. 9. shows the proposed VGG 19 Architecture

D. *Proposed Methodology*

The proposed method is divided into four stages: dataset preparation, machine learning models with hyperparameter tuning, classification of QR codes using deep learning models like Custom Made CNN, Resnet 50, and VGG 19, and finally, detection of malicious QR codes using Python's Tkinter module.

Preparing the dataset for both machine learning and deep learning models is the main goal of the initial step. This includes using the malicious URLs found in the database to create a QR dataset.

The hyperparameters of machine learning models are tweaked in the second step to attain the best performance. In the parameter tuning sub-section, the specifics of the parameter adjustment are covered.

The third stage involves binary classification of QR codes using deep learning models, notably Custom Made CNN, Resnet 50, and VGG 19. These models have a reputation for excelling at picture categorization tasks. There are mainly seven classification tasks that were performed, namely benign and Malware, benign and spam, benign and defacement, benign and phishing, malware and spam, malware and phishing, malware and defacement.

The Python Tkinter library is used to identify malicious QR codes at the final stage. A popular GUI toolkit that enables the building of straightforward and understandable interfaces is Tkinter.

Overall, the strategy comprises using both machine learning and deep learning approaches to solve the issue of detecting malicious URLs and QR codes, respectively, and combining them with a user-friendly interface using the Tkinter module.

V. OBSERVATIONS

A. *Challenges Faced*

Malware, also termed malicious software, poses many difficulties for cybersecurity. First and foremost, malware seriously jeopardizes the integrity and security of computer systems and networks. It can protect systems using various techniques, including malicious websites, email attachments and software flaws. Malware is capable of carrying out a variety of malicious tasks once it has gained access to a system, such as stealing confidential data like passwords or financial information, corrupting or erasing files, interfering with system performance, or even allowing hackers unauthorized access. These behaviors may result in monetary losses, privacy violations, and serious business operations disruptions. Furthermore, the threat landscape is always shifting, making it difficult for security professionals to stay ahead of it and effectively combat it. New variants and techniques of malware are constantly emerging.

Second, dealing with malware necessitates a multifaceted strategy that includes proactive detection, prevention, and response techniques. To reduce the risk of malware infections, effective cybersecurity measures must be developed and put into place. In order to do this, it is mandatory to implement trustworthy antivirus and anti-malware software, update operating systems and software programs frequently to fix vulnerabilities, and inform users about safe online conduct, such as avoiding dubious downloads and clicking on unknown links. Malware can still enter systems, though, despite the precautions that are in place. Therefore, to quickly identify and mitigate malware threats, organizations must also invest in advanced detection mechanisms like IDS (Intrusion Detection System) and behavioral analysis tools. Additionally, in order to lessen the effects of malware attacks and speed up the recovery of compromised systems, organizations must create effective incident response plans.

In conclusion, malware poses a broad range of difficulties for cybersecurity. It is a persistent and constantly evolving threat due to its capacity to infiltrate systems, engage in malicious actions, and evolve. A thorough and proactive strategy is needed to combat malware, including preventive measures, cutting-edge detection technologies, and effective incident response techniques. Individuals and organizations can better guard themselves against the harmful effects of malware by remaining vigilant, routinely updating defenses, and investing in effective cybersecurity practices.

B. *Optimal Solution*

Due to their capacity to analyze massive amounts of data and spot patterns or anomalies that are suggestive of malware

Authorized licensed use limited to: UNIVERSITI TEKNOLOGI MARA. Downloaded on April 28,2025 at 03:36:16 UTC from IEEE Xplore. Restrictions apply.

behavior, machine learning detectors present a promising method for malware detection. The capability of machine learning to learn from historical data and modify its detection abilities over time is one of its key advantages. The machine learning model can learn to distinguish between benign and malicious files based on the patterns and features it has identified by being trained on a diverse dataset of known malware samples and legitimate software. As a result, machine learning detectors are very good at picking up previously unknown or zero-day malware because they can spot resemblances or patterns that correspond to well-known malware traits.

Furthermore, automated detection methods using machine learning detectors can eliminate the need for manual analysis and enable real-time detection. Machine learning models can continuously learn and adapt to new threats without the need for manual intervention, in contrast to traditional signature-based methods that require regular updates to include new malware signatures. Machine learning detectors are especially useful in the quick-changing malware landscape because of their adaptability. A thorough approach to detecting malware is made possible by the application of machine learning techniques to a variety of data sources, such as file analysis, network traffic analysis, or behavior monitoring.

It is important to remember that machine learning detectors have limitations and are not infallible. Machine learning models may be severely hampered by adversarial attacks, in which malware authors purposefully alter or obfuscate their code to avoid detection. Furthermore, because machine learning detectors rely on historical data, they might have trouble spotting new, sophisticated malware that displays novel behaviors. To ensure the efficiency of machine learning detectors and to stay one step ahead of evolving malware threats, regular updates and continuous monitoring are required.

Deep learning techniques like CNN, ResNet, and VGG are used to classify if a QR code is malicious or not, but the best strategy will rely on a variety of criteria. Here are some justifications for why these techniques might work for classifying QR codes:

The capture of complicated visual patterns and learning of hierarchical representations from the data makes CNN, ResNet, and VGG ideal for picture classification applications. Models of Deep learning can efficiently collect the encoded information found in QR codes, which includes various patterns, forms, and alignments.

Transfer learning capabilities: ResNet and VGG CNN architectures are frequently pre-trained on huge picture datasets like ImageNet. The models can learn general properties like edges, textures, and item forms thanks to this pre-training. Even with a relatively limited dataset, you may use transfer learning to modify these pre-trained models for the particular purpose of QR code categorization.

Resilience against variations: Deep learning models like ResNet, CNN, and VGG have demonstrated resilience against changes in lighting, orientations, and noise, which can be present in QR codes. Even in the presence of various variations, their ability to generalize well and categorize QR codes accurately is aided by their capacity to learn from a wide number of distinct samples.

Scalability: Deep learning models are appropriate for situations when you need to classify a large number of QR codes since they can handle enormous amounts of data efficiently. If you have the required computing power, you can train and use deep learning models to process QR codes at scale.

While CNN, ResNet, and VGG can all be useful options, the best option will rely on the particular specifications of the task for classifying QR codes. When choosing the optimal model for any application, one must take into account the elements like the quantity and quality of the dataset, the available computer resources, and the desired trade-off between computational efficiency and accuracy. To determine the best deep learning strategy for a particular use case, testing, and tweaking are frequently required.

VI. Comparison of Results

A. *Existing Models Study:*

Machine learning and deep learning have been used to develop a number of models for the detection of harmful URLs and QR codes. The architecture, input features, and performance of these models vary.

The Random Forest model suggested by Ma et al. is one illustration of a previously created model for malicious URL identification. (2014). Lexical and host-based features were combined with this model to train the classifier, which resulted in an accuracy of 96.23% on the dataset employed.

For malicious URL detection, Zhang et al.'s CNN is a more current deep learning-based model. (2021). Character-level embedding of URLs and a one-dimensional CNN architecture were employed in this model's input features. This model beat earlier models, with a dataset accuracy of 98.63%.

A previously created model for detecting QR codes is the SVM classifier suggested by Chen et al. (2014). In order to train the classifier, this model included geometric and statistical information derived from QR codes. The model's accuracy on the dataset they utilized was 98.6%.

In contrast, Lee et al.'s Faster R-CNN model is a current deep learning-based model for QR code detection. (2021). To recognize and categorize QR codes, this model utilized an object identification architecture based on CNN. The dataset they used yielded a 99.3% accuracy for their model.

In general, deep learning-based models have outperformed conventional machine learning models in comparison to detecting harmful URLs and QR codes. Furthermore, deep learning-based models can learn pertinent features automatically from the data, obviating the requirement for human feature engineering. Deep learning models, however, need very lot of data and computing power, which may restrict how saleable they may be in practical situations.

B. *Results Obtained*

Authorized licensed use limited to: UNIVERSITI TEKNOLOGI MARA. Downloaded on April 28,2025 at 03:36:16 UTC from IEEE Xplore. Restrictions apply.

TABLE I. SHOWS METRICS COMPARISON OF DIFFERENT MODELS IN ALL FEATURES DATASET

	Accuracy	Precision	Recall	F1 score	Kappa
XGBoost	0.937	0.938	0.937	0.937	0.92
Random Forest	0.965	0.966	0.965	0.966	0.956
Decision Tree (entropy)	0.94	0.94	0.94	0.94	0.924
Decision Tree (gini)	0.933	0.933	0.933	0.933	0.915
Gaussian naïve Bayes	0.456	0.5	0.456	0.375	0.318

TABLE II. SHOWS METRICS COMPARISON OF ALL BEST FEATURES

	Accuracy	Precision	Recall	F1 score	Kappa
Logistic Regression (Newton–cg)	0.582	0.594	0.582	0.565	0.475
Random Forest (entropy)	0.929	0.929	0.929	0.929	0.911
Random Forest (gini)	0.932	0.932	0.932	0.932	0.915
Decision Tree (entropy, best)	0.803	0.804	0.803	0.803	0.753
Decision Tree (entropy, random)	0.595	0.624	0.595	0.577	0.494
Decision Tree (gini, best)	0.795	0.8	0.795	0.795	0.743
Decision Tree (gini, random)	0.738	0.739	0.738	0.738	0.672

TABLE III. SHOWS METRICS COMPARISON OF ALL_INFO GAIN FEATURES

	Accuracy	Precision	Recall	F1 score	Kappa
Random Forest	0.959	0.959	0.959	0.959	0.948
XGBoost	0.966	0.966	0.966	0.966	0.957
Decision Tree (gini)	0.809	0.81	0.809	0.809	0.761
Decision Tree (entropy)	0.822	0.824	0.822	0.821	0.778
Light GBM	0.954	0.954	0.954	0.954	0.942

TABLE IV. SHOWS METRICS COMPARISON OF DEEP LEARNING MODELS IMPLEMENTED IN QR DATASET

Comparison	**Our Model (CNN)**	**Resnet 50**	**VGG 19**
Benign & Malware	98.87	79.52	64.04
Benign & Spam	94.63	95.63	92.88
Benign & Defacement	1	1	1
Benign & Phishing	97.13	86.65	74.18
Malware & Spam	91.88	83.27	79.52
Malware & Defacement	87.39	95.63	68.16
Malware & Phishing	85.53	87.53	79.67

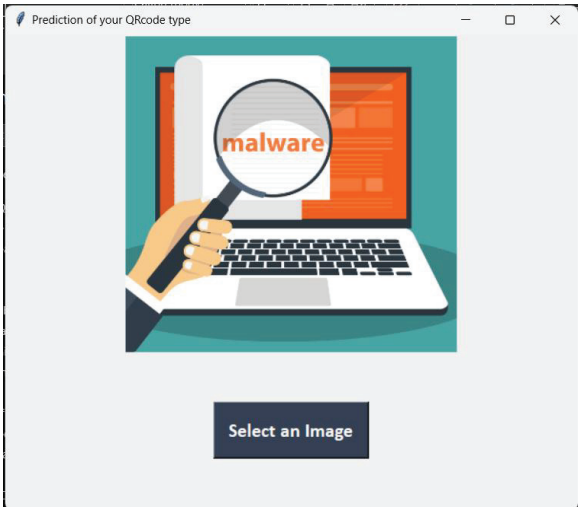

Fig. 10. (a)

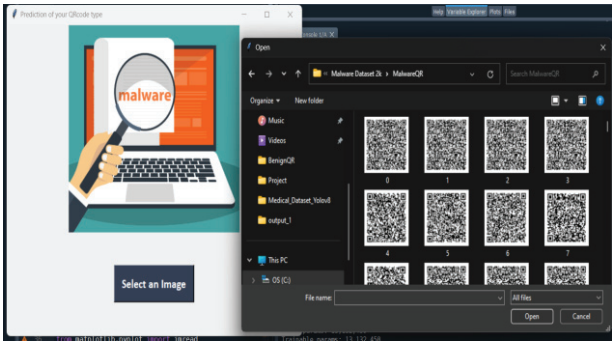

Fig 10 (b)

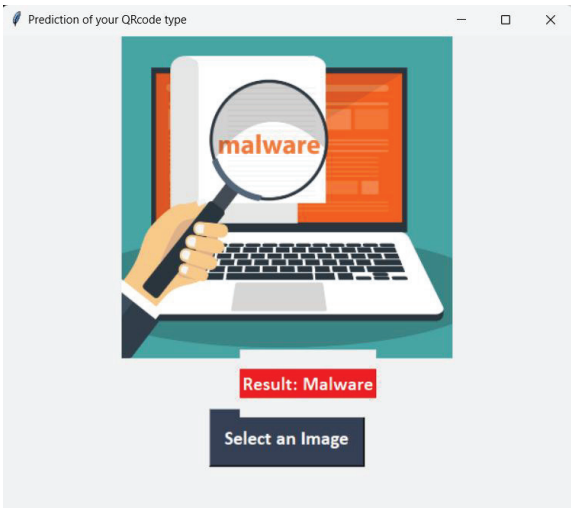

Fig 10 (c)

Fig 10 (a) (b) and (c) shows the working of the Tkinter Graphical User Interface

VII. CONCLUSION

We implemented six different Machine Learning models as a part of our research on the URL datasets, namely dataset

Authorized licensed use limited to: UNIVERSITI TEKNOLOGI MARA. Downloaded on April 28,2025 at 03:36:16 UTC from IEEE Xplore. Restrictions apply.

with all features, dataset with best fit features and the info gain dataset. Out of all the ML models the XG Boost and the Random Forest model outperformed the other models by giving an accuracy of 96% and 95%, respectively. The custom built CNN model outperforms the VGG and Resnet model while comparing Benign and Malware, Benign and defacement, Benign and Phishing, Malware and Spam by giving an accuracy of 98.87, 1, 97.13, 91.88, respectively.

Real-time detection can be made possible by automated detection techniques that use machine learning detectors, which can do away with the requirement for manual analysis. In contrast to traditional signature-based approaches, which necessitate routine updates to include new malware signatures, machine learning models may continually learn and adapt to new threats without the need for user intervention. Because of their versatility, machine learning detectors are particularly helpful in the rapidly changing and malware field. It is critical to remember that machine learning detectors have flaws and are not perfect. Adversarial assaults, in which malware authors intentionally change or obscure their code to escape detection, may seriously hinder machine learning models. As a result of their reliance on historical data, machine learning detectors may also have problems identifying new, sophisticated malware that exhibits unexpected characteristics. Regular upgrades and ongoing monitoring are necessary to guarantee the efficiency of machine learning detectors and to stay ahead of evolving malware threats.

CNN, ResNet, and VGG are among the best models for image classification applications because they can capture intricate visual patterns and develop hierarchical representations from the data. The encoded data included in QR codes, which comprises a variety of patterns, shapes, and alignments, can be easily collected using deep learning models. While CNN, ResNet, and VGG are all viable possibilities, the most effective choice will depend on the unique requirements of the QR code classification task. One must consider factors including the size and quality of the dataset, the available computer resources, and the desired trade-off between computational efficiency and accuracy when selecting the best model for any application. Testing and tweaking are frequently needed to find the optimum deep learning approach for a certain use case.

VIII. Declaration

This research work does not involve any grants or financial support by any government or non-government organizations and is a completely a non funded research done based on the interest of the authors.

We the authors declare that there is no competing interests or other interests that might be perceived to influence the results and/or discussion reported in this paper.

We declare that the results/data/figures in this manuscript have not been published elsewhere. All the materials is owned by the authors and no permission is required. Our initial dataset is taken from a open source dataset [11] and no external permission is required for the same.

This research work does not contain any studies involving human or animal participants performed by any of the authors.

There is no conflicts of interests to disclose.

References

[1] Mohammad Saiful Islam Mamun, Mohammad Ahmad Rathore, Arash Habibi Lashkari, Natalia Stakhanova and Ali A. Ghorbani, "Detecting Malicious URLs Using Lexical Analysis", Network and System Security, Springer International Publishing, pp. 467-482, 2016.

[2] Johnson, C., Khadka, B., Basnet, R. B., & Doleck, T. (2020). Towards Detecting and Classifying Malicious URLs Using Deep Learning. J. Wirel. Mob. Networks Ubiquitous Comput. Dependable Appl., 11(4), 31-48.

[3] Chow, Y. W., Susilo, W., Wang, J., Buckland, R., Baek, J., Kim, J., & Li, N. (2021). Utilizing QR codes to verify the visual fidelity of image datasets for machine learning. Journal of Network and Computer Applications, 173, 102834.

[4] Do Xuan, C., Nguyen, H. D., & Tisenko, V. N. (2020). Malicious URL detection based on machine learning. International Journal of Advanced Computer Science and Applications, 11(1).

[5] Pawar, A., Fatnani, C., Sonavane, R., Waghmare, R., & Saoji, S. (2022, August). Secure QR Code Scanner to Detect Malicious URL using Machine Learning. In 2022 2nd Asian Conference on Innovation in Technology (ASIANCON) (pp. 1-8). IEEE.

[6] Abusitta, A., Li, M. Q., & Fung, B. C. (2021). Malware classification and composition analysis: A survey of recent developments. Journal of Information Security and Applications, 59, 102828.

[7] Awan, M. J., Masood, O. A., Mohammed, M. A., Yasin, A., Zain, A. M., Damaševičius, R., & Abdulkareem, K. H. (2021). Image-based malware classification using VGG19 network and spatial convolutional attention. Electronics, 10(19), 2444.

[8] Agarap, A. F. (2017). Towards building an intelligent anti-malware system: a deep learning approach using support vector machine (SVM) for malware classification. arXiv preprint arXiv:1801.00318.

[9] Raff, E., & Nicholas, C. (2020). A survey of machine learning methods and challenges for windows malware classification. arXiv preprint arXiv:2006.09271.

[10] Ullah, F., Alsirhani, A., Alshahrani, M. M., Alomari, A., Naeem, H., & Shah, S. A. (2022). Explainable malware detection system using transformers-based transfer learning and multi-model visual representation. Sensors, 22(18), 6766.

[11] Mohammad Saiful Islam Mamun, Mohammad Ahmad Rathore, Arash Habibi Lashkari, Natalia Stakhanova and Ali A. Ghorbani, "Detecting Malicious URLs Using Lexical Analysis", Network and System Security, Springer International Publishing, pp. 467-482, 2016.

[12] Royal, P., Halpin, M., Dagon, D., Edmonds, R., & Lee, W. (2006, December). Polyunpack: Automating the hidden-code extraction of unpack-executing malware. In 2006 22nd Annual Computer Security Applications Conference (ACSAC'06) (pp. 289-300). IEEE.

[13] Nataraj, L., Karthikeyan, S., Jacob, G., & Manjunath, B. S. (2011, July). Malware images: visualization and automatic classification. In Proceedings of the 8th international symposium on visualization for cyber security (pp. 1-7).

Authorized licensed use limited to: UNIVERSITI TEKNOLOGI MARA. Downloaded on April 28,2025 at 03:36:16 UTC from IEEE Xplore. Restrictions apply.